AMATH 482: HOMEWORK 5

DORIAN RAMEY

AMATH, University of Washington, Seattle, WA
dramey2@uw.edu

Abstract. In this report, we describe the methods used to build, modify, and test a Fully Connected Neural Network for the purposes of classifying images in the FashionMNIST dataset, a dataset made of images of various clothing items. In order to achieve the highest accuracy, we tested various optimization and initialization functions as well as with and without batch normalization, recording and comparing their training losses, validation accuracies, and testing accuracies.

1. Introduction and Overview

In this project, we build and test a Fully Connected Deep Neural Network (FCN) to classify images from the FashionMNIST dataset. FashionMNIST is a dataset consisting of a training set, with 60K images, and a testing set, consisting of 10K images. Each image is a grayscale 28 x 28 image of a clothing item. After building a baseline model, we will test different variations of this model by changing optimizer, initialization, and normalization to see if we can improve the model's training time and accuracy.

2. Theoretical Background

2.1. **Fully Connected Deep Neural Networks.** A Fully Connected Deep Neural Network, or FCN, is a model for classification tasks that attempts to mimic the workings of the human brain by using a series of connected "neurons". Classification is a supervised learning method that takes input, applies some function to it, and attempts to label it. Some data types are complex, such as the images in the FashionMNIST dataset, and traditional regression methods do not achieve high accuracy, so it is necessary to use an FCN.

An FCN is built of neurons which take an input and apply an activation function. These neurons are arranged in layers. The first layer takes the input, then there are a series of hidden layers. The hidden layers transform the input through learned computations. Each layer may have a different number of neurons, but the neurons of each layer are connected to the next layer. Finally, there is an output layer. At this layer, no further non-linearity is applied. Instead, the output layer returns scores for each possible classification. A function, such as the softmax function, converts these scores into probabilities representing how likely it is that a sample fits each classification.

2.1.1. *Training.* Training of an FCN consists of phases. The first step is to shuffle the data and separate it into approximately equally sized batches. Then for each batch we do a forward pass and a backward pass.

In the forward pass, we calculate our output based on the current model weights. After this step, we have a preliminary output, which is then used to calculate loss and move into the backward pass.

Date: March 20, 2026.

After the forward pass has been completed, we do a backward pass. This pass is where the model "learns" from what happened in the forward pass. The loss and gradients, calculated using partial derivatives, are used to determine how much each weight contributed to the loss. Then, adjustments are made to the weights to try to reduce the loss and therefore improve model performance [2].

2.1.2. *Optimizers.* Optimizers are the functions that work to reduce the model loss by finding the global minima. One example of an optimizer is stochastic gradient descent, or SGD. This optimizer aims to move along the loss function to reach the lowest points by iteratively updating the model weights based on partial derivatives. For this technique to be successful, it is important to find a learning rate which allows the loss to drop quickly and then stabilize. Learning rates that are too high or low may cause a lack of convergence or for the model to run for a long time without finding the minima. Other more complex methods include Adam and RMSprop. In these methods, not only the model weights, but the learning rate changes throughout the training process [1].

2.1.3. *Batch Normalization.* Sometimes, issues can occur such as gradient explosion or vanishing or extremely long run times. To prevent this, the method of batch normalization was created. This introduces an additional normalization step before the activation function is applied. In doing so, batch normalization stabilizes the training process [3].

2.2. **Training Loss.** The training loss is a metric for determining how well the model is able to classify the data. It is equal to the sum of errors made by the model and is measured after each batch. Our goal in training our neural net is to have a low training loss value, and the optimizer functions we use are informed by the training loss in making adjustments to the model weights.

3. Algorithm Implementation and Development

In the template provided, our first step of reading the FashionMNIST dataset and separating it into training and test sets has been done for us. Therefore, we move on to developing the neural network.

3.1. **Baseline Neural Network.** First, we developed a baseline neural network. To do this, we initialized our network with hidden layers and ReLU as the activation function.

3.2. **Training Model.** In order to train the model, we needed to create a loop that would iterate through a given number of epochs, calculating the loss and validation accuracy after each epoch and adjusting the model weights based on these values. The first step in the loop was to convert the images into vectors which were used as calculate training loss. Then back propagation and optimization were performed. Finally, the validation accuracy was calculated.

We used this loop to test our model and make adjustments to the hyperparameters: learning rate, number of epochs, batch size, and number of hidden layers. We did this until we achieved a model with at least 85 percent validation accuracy.

3.3. **Adjusting and Testing.** Once we had a baseline model, we tested various modifications one at a time to see if they had any improvements on our model's accuracy and training loss. First, we adjusted the optimization function. We tested Adam and RMSprop methods. Then, we used the one that achieved the smallest loss and adjusted the initialization. We used random normal, Xavier normal, and Kaiming uniform initializations. Once more, we kept the one that achieved the lowest training loss. Finally, we tested with batch normalization.

For each of these changes, we recorded the validation accuracy and training loss, created a plot for the validations accuracy and training loss, and calculated and recorded the test accuracy so that we could fully compare the results of the methods.

4. Computational Results

We achieved a baseline model with 20 epochs, a learning rate of 0.01, and two hidden layers. Building on this, we tested RMSprop and Adam optimization methods and found that the optimization we used originally, stochastic gradient descent (SGD) was best suited to our data as it achieved the highest accuracy, so we continued with this as we tested initialization functions. However, the training loss between Adam and the baseline were very similar, while RMSprop had the highest training loss. The initializations we tried were random normal, Xavier normal, and Kaiming uniform. We found that Kaiming uniform had the lowest loss so we kept that and added batch normalization for our final test. While both Xavier normal and Kaiming uniform caused a decrease in testing loss, Kaiming normal had a slightly lower loss. Random normal had very little change from the baseline model. We found that with batch normalization, we achieved a far lower testing loss than any of the models without this normalization. The normalized model also had a higher test accuracy than most other models, although it was slightly lower than the baseline and xavier normal models. These findings are shown in Table 1 1.

model	training loss	validation accuracy	test accuracy
baseline	0.3342	88.93	88.39
Adam	0.3182	84.78	83.82
RMSprop	0.5156	85.03	83.87
Random Normal	0.3875	87.75	87.25
Xavier Normal	0.1773	89.71	88.84
Kaiming Uniform	0.1226	88.63	87.69
Batch Normalization	0.05280	89.56	87.99

Table 1. Table showing the training loss, validation accuracy, and test accuracy for the baseline model and each of the modifications made.

We also created plots for each of these models. The loss plots are shown in Figure One 1. The validation accuracy plots are shown in Figure Two 2.

5. Summary and Conclusions

In summary, we created a baseline Fully Connected Neural Network to classify images from the FashionMNIST dataset. We then tested modifications on this model and found that Stochastic Gradient Descent optimization with Kaiming uniform initialization and batch normalization created the lowest training loss while keeping a high validation and testing accuracy. In future models, we can use these as guidelines when working with similar datasets or use this model as a starting point for testing further modifications.

6. Acknowledgments

The author would like to thank Professor Frank for providing the code for reading the data and the template for the FCN.

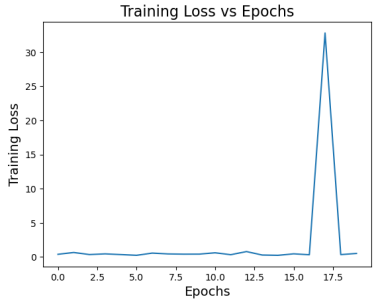
References

[1] Types of optimizers in deep learning: A comprehensive guide. `https://medium.com/@sushmita2310/types-of-optimizers-in-deep-learning-a-comprehensive-guide-af258d6acf66`. Accessed: 03-19-2026.

[2] Understanding neural network: Forward and backward passes. `https://medium.com/@tensor.tales/understanding-neural-network-forward-and-backward-passes-explained-a3fa7341def6`. Accessed: 03-19-2026.

[3] I. P. Carrascosa. A gentle introduction to batch normalization.

(A) Baseline model



(B) RMSprop as optimization

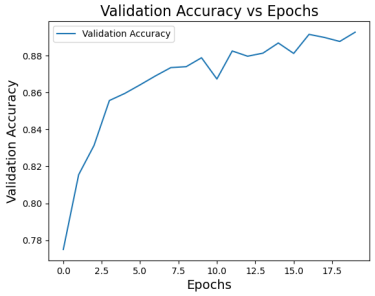
(C) Random Normal Initialization

(D) Xavier Normal Initialization

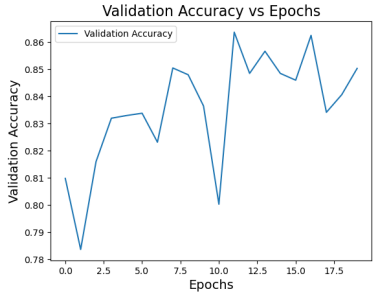
(E) Kaiming Uniform Initialization

(F) With Batch Normalization

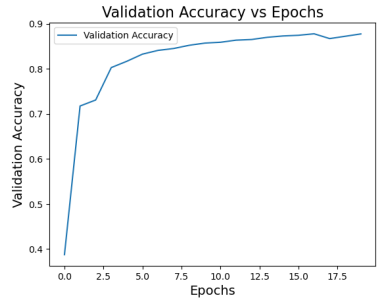
FIGURE 1. Training Loss plots for each of the models tested.



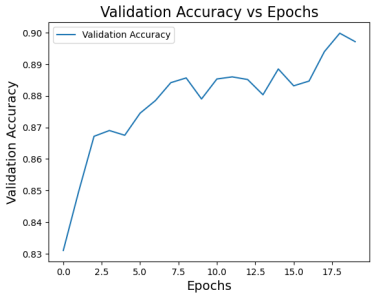
(A) Baseline model



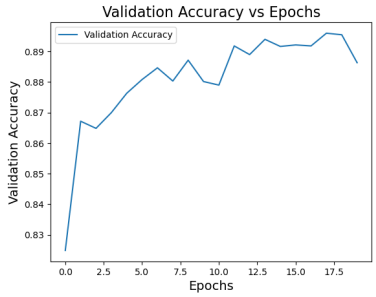
(B) RMSprop as optimization



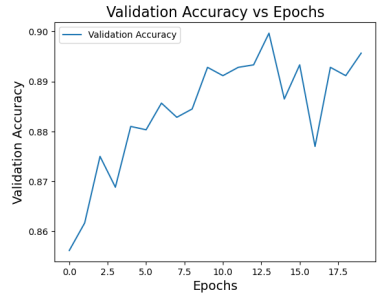
(C) Random Normal Initialization



(D) Xavier Normal Initialization



(E) Kaiming Uniform Initialization



(F) With Batch Normalization

FIGURE 2. Training Loss plots for each of the models tested.